

Lab 4: Vibe Coding – Maze Chase

Name: Anupriya

SRN: PES2UG24CS074

SECTION: 5B

Objective

Use an LLM as a coding assistant to inspect the existing Maze Chase project, identify defects, and implement the required features.

Use Vibe Coding prompts to make the changes efficiently and test the final solution.

CHAT HISTORY

Initial Analysis

Project execution: The existing Maze Chase project was run using the project setup and Python environment. The game launched successfully, and the existing maze, player, one-enemy BFS chase, exit, collision, and restart behavior were inspected before feature changes.

Existing structure reviewed: The relevant files were game/game_engine.py and game/entities.py. The Enemy class already used BFS pathfinding to move toward the player. The game engine initially managed a single enemy.

Task 1: Multiple Enemies

Vibe Coding Prompt

In my maze-chase pygame game, I have one Enemy that uses BFS to chase the player. Add two more Enemy instances starting at different maze corners. Each should independently call bfs() on every update. List them in a self.enemies list and loop over them.

Requirements:

- Keep the existing behavior.
- Three enemies should start at different maze corners.
- Each enemy should independently chase the player using the existing BFS logic.
- Collision with any enemy should cause game over.
- Do not implement Tasks 2–4.
- Make focused/minimal changes only.

Implementation / Changes

Implemented Task 1 in game/game_engine.py:

- Replaced the single self.enemy with self.enemies containing three Enemy objects.
- Enemies start at bottom-right, top-right, and bottom-left.
- Looped through self.enemies for enemy updates and drawing.
- Checked player collision against every enemy.
- Existing Enemy BFS logic was retained.

Testing Result

Test result: Three enemies appeared and independently chased the player. Task 1 was confirmed working.

Task 2: Speed Up Over Time

Vibe Coding Prompt

Add a difficulty ramp to my maze-chase game. Track elapsed time with `pygame.time.get_ticks()`. Every 15 seconds, reduce `enemy.move_interval` by 2 (minimum 5). Apply the speed increase to all three enemies. Show the current enemy speed tier in the HUD.

Requirements:

- Keep Task 1 working.
- Do not implement Tasks 3–4.
- Keep player movement, maze, exit, restart, and collision behavior unchanged.
- Make focused/minimal changes only.

Implementation / Changes

Implemented Task 2 in game/game_engine.py:

- Added `start_time` using `pygame.time.get_ticks()`.
- Added `speed_tier` starting at 1.
- Every 15 seconds, the tier increases.
- Enemy `move_interval` is reduced by 2 for every new tier, with a minimum of 5.
- The new interval is applied to all three enemies.
- HUD displays the current Speed Tier.

Testing Result

Test result: HUD initially showed Speed Tier: 1 and changed to Speed Tier: 2 after 15 seconds. The enemies became faster.

Task 3: Power Pellet (Freeze Enemy)

Vibe Coding Prompt

In my existing Python Pygame maze-chase game, implement ONLY Task 3: Power Pellet (Freeze Enemy).

Current state:

- Task 1 is already implemented: there are 3 Enemy instances stored in self.enemies.
- Task 2 is already implemented: enemy speed increases every 15 seconds using `pygame.time.get_ticks()`, and the HUD shows the current Speed Tier.
- Do NOT remove or break Tasks 1 or 2.
- Do NOT implement Task 4 (score) yet.

Task 3 requirements:

1. Add one power pellet to the maze as a visible yellow circle.
2. Place it somewhere reachable by the player.
3. When the player's rect collides with the power pellet:
 - The pellet should disappear/become collected.
 - Freeze ALL 3 enemies for 5 seconds.
4. Each Enemy should have a frozen state and a 300-frame freeze countdown.
5. While an enemy is frozen, its `update()` must not move it or perform BFS movement.
6. After the 300-frame countdown finishes, the enemies should automatically resume chasing the player normally.
7. Show a clear visual frozen indicator on each frozen enemy.
8. The existing Task 2 speed ramp must continue to work after the freeze ends.
9. Keep existing player movement, maze, exit, restart, and collision behavior unchanged.
10. Pressing R should reset the pellet and the frozen state.
11. Make focused/minimal changes only.

Implementation / Changes

Implemented Task 3 in `game/entities.py` and `game/game_engine.py`:

- Added frozen and freeze_timer state to Enemy.
- While frozen, `Enemy.update()` counts down the 300-frame timer and returns without performing BFS movement.
- After the timer reaches zero, the enemy automatically resumes normal chasing.
- Added a visible frozen appearance/indicator.
- Added one reachable yellow power pellet.
- When the player collides with the pellet, it disappears and all three enemies are frozen for 300 frames.
- Reset restores the pellet and enemy states.
- Existing Task 1 and Task 2 behavior was preserved.

Testing Result

Test result: The yellow pellet appeared, disappeared when collected, and all three enemies froze and then resumed movement after approximately 5 seconds.

Task 4: Survival Score

Vibe Coding Prompt

In my existing Python Pygame maze-chase game, implement ONLY Task 4: Score by Distance / Survival Score.

Current state:

- Task 1 is completed: there are 3 Enemy instances stored in self.enemies.
- Task 2 is completed: enemy speed increases every 15 seconds using `pygame.time.get_ticks()`, and the HUD shows the current Speed Tier.
- Task 3 is completed: a power pellet freezes all 3 enemies for 300 frames (5 seconds), then they resume chasing.
- Do NOT remove or break Tasks 1, 2, or 3.

Task 4 requirements:

1. Add a survival score variable called `self.score`.
2. Initialize `self.score = 0` when the game is reset.
3. Increment `self.score` by 1 every frame while the player is alive and the game is active.
4. Display the survival time in the HUD as "Survived: Xs", where $X = \text{self.score} // 60$.
5. Keep the existing Speed Tier display in the HUD.
6. When the player is caught, show the final survival score/time on the game-over overlay.
7. If the player reaches the EXIT, also show the survival score/time on the escaped overlay.
8. Pressing R must reset the score to 0.
9. Keep all existing game behavior unchanged.
10. Make focused/minimal changes only.

Implementation / Changes

Implemented Task 4 in `game/game_engine.py`:

- Added `self.score = 0` during reset.
- Incremented `self.score` by 1 on every active game frame.
- Displayed survival time using `self.score // 60`.
- Kept the Speed Tier in the HUD.
- Added the final survival time to both CAUGHT and ESCAPED overlays.
- Restart resets the score to zero.
- HUD wording was shortened to fit the existing game window.

Testing Result

Test result: Survival time increased while the player was alive, and the final survival time was shown on the game-over/escaped overlay. Restart reset the score.

Representative Vibe Coding Prompts Used

- 1:** Inspect the existing Maze Chase game and add two more Enemy instances using the existing BFS chase logic. Keep the changes focused and do not implement the later tasks.
- 2:** Add a difficulty ramp using `pygame.time.get_ticks()`. Every 15 seconds reduce `enemy.move_interval` by 2, minimum 5, apply it to all enemies, and show the current speed tier in the HUD.
- 3:** Implement only the Power Pellet task: add a reachable yellow pellet, freeze all three enemies for 300 frames when collected, prevent BFS movement while frozen, show a frozen indicator, and restore normal chasing afterward.
- 4:** Implement only the Survival Score task: add `self.score`, increment it once per active frame, display Survived: Xs using `score // 60`, retain Speed Tier, and show the final survival time on caught/escaped overlays.
- 5:** Review the final implementation against the README checklist and suggest focused test cases for all four tasks without rewriting unrelated code.